



Εθνικό Μετσόβιο Πολυτεχνείο
Σχολή Ηλεκτρολόγων Μηχανικών και Μηχανικών Υπολογιστών

Ψηφιακά Συστήματα VLSI

3η Εργαστηριακή Άσκηση
Σχεδίαση Μονάδων Υλικού με την Τεχνική Pipelining

ΟΝΟΜΑΤΕΠΩΝΥΜΑ	Ευαγγελία Αγγελάκη (el21037) Μαρία Φατούρου (el21077)
ΕΞΑΜΗΝΟ	8ο
ΟΜΑΔΑ	<u>Βάρδια Β/Ομάδα 17</u>

Σχεδίαση Μονάδων Υλικού με την Τεχνική Pipelining

1. Υλοποίηση ενός σύγχρονου πλήρη αθροιστή (Full Adder - FA) με περιγραφή συμπεριφοράς (Behavioral).

Ο πλήρης αθροιστής:

```
library IEEE;
```

```
use IEEE.STD_LOGIC_1164.ALL;
```

```
entity full_adder1 is
```

```
Port ( A, B, Cin : in STD_LOGIC;
```

```
Sum, Cout : out STD_LOGIC);
```

```
end full_adder1;
```

```
architecture Behavioral of full_adder1 is
```

```
begin
```

```
    process(A, B, Cin)
```

```
    begin
```

```
        if (A ='0' and B ='0' and Cin ='0') then
```

```
            Sum <='0';
```

```
            Cout <='0';
```

```
        elsif (A ='0' and B ='0' and Cin ='1') then
```

```
            Sum <='1';
```

```
            Cout <='0';
```

```
        elsif (A ='0' and B ='1' and Cin ='0') then
```

```
            Sum <='1';
```

```
            Cout <='0';
```

```
        elsif (A ='0' and B ='1' and Cin ='1') then
```

```
            Sum <='0';
```

```
            Cout <='1';
```

```
        elsif (A ='1' and B ='0' and Cin ='0') then
```

```
            Sum <='1';
```

```
            Cout <='0';
```

```
        elsif (A ='1' and B ='0' and Cin ='1') then
```

```
            Sum <='0';
```

```
            Cout <='1';
```

```
        elsif (A ='1' and B ='1' and Cin ='0') then
```

```
            Sum <='0';
```

```
            Cout <='1';
```

```
        elsif (A ='1' and B ='1' and Cin ='1') then
```

```
            Sum <='1';
```

```
            Cout <='1';
```

```
        else Sum<='U';
```

```
            Cout<='U';
```

```
        end if;
```

```
    end process;
```

```
end Behavioral;
```

To test bench του πλήρους αθροιστή:

```
library IEEE;  
use IEEE.STD_LOGIC_1164.ALL;
```

```
entity full_adder_tb is  
-- Port ( );  
end full_adder_tb;
```

architecture Behavioral of full_adder_tb is

```
component full_adder  
    port( A,B,Cin: in std_logic;  
          Cout,sum: out std_logic);  
end component;  
signal A_tb: std_logic;  
signal B_tb: std_logic;  
signal Cin_tb:std_logic;  
signal Cout_tb:std_logic;  
signal sum_tb:std_logic;
```

```
begin  
DUT: full_adder port map(A_tb, B_tb, Cin_tb, Cout_tb, Sum_tb);  
stimulus:process  
begin
```

```
    A_tb<='0';  
    B_tb<='0';  
    Cin_tb<='0';  
    wait for 20ns;
```

```
    A_tb<='0';  
    B_tb<='0';  
    Cin_tb<='1';  
    wait for 20ns;
```

```
    A_tb<='0';  
    B_tb<='1';  
    Cin_tb<='0';  
    wait for 20ns;
```

```
    A_tb<='0';  
    B_tb<='1';  
    Cin_tb<='1';  
    wait for 20ns;
```

```
    A_tb<='1';  
    B_tb<='0';  
    Cin_tb<='0';  
    wait for 20ns;
```

```

A_tb<='1';
B_tb<='0';
Cin_tb<='1';
wait for 20ns;

```

```

A_tb<='1';
B_tb<='1';
Cin_tb<='0';
wait for 20ns;

```

```

A_tb<='1';
B_tb<='1';
Cin_tb<='1';
wait for 20ns;

```

```

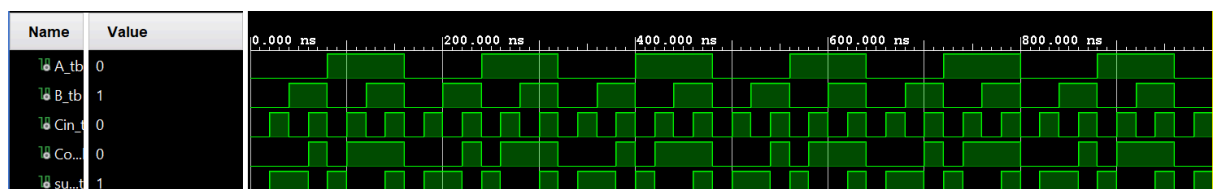
end process;
end Behavioral;

```

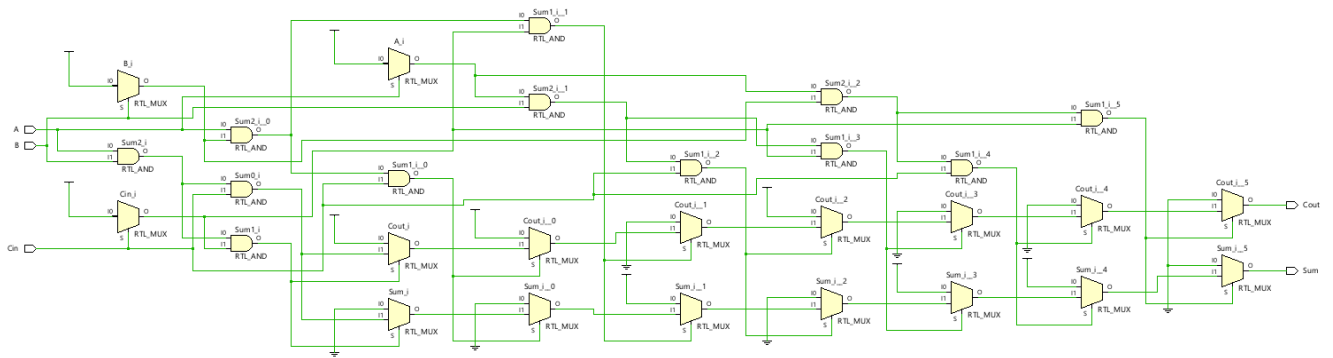
Ο πίνακας αληθείας του πλήρους αθροιστή:

Inputs			Outputs	
A	B	C _{in}	Sum	Carry
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Η **προσομοίωση** του κώδικα πλήρους αθροιστή που παραθέσαμε παραπάνω ταυτίζεται με τον πίνακα αληθείας:



To RTL schematic:



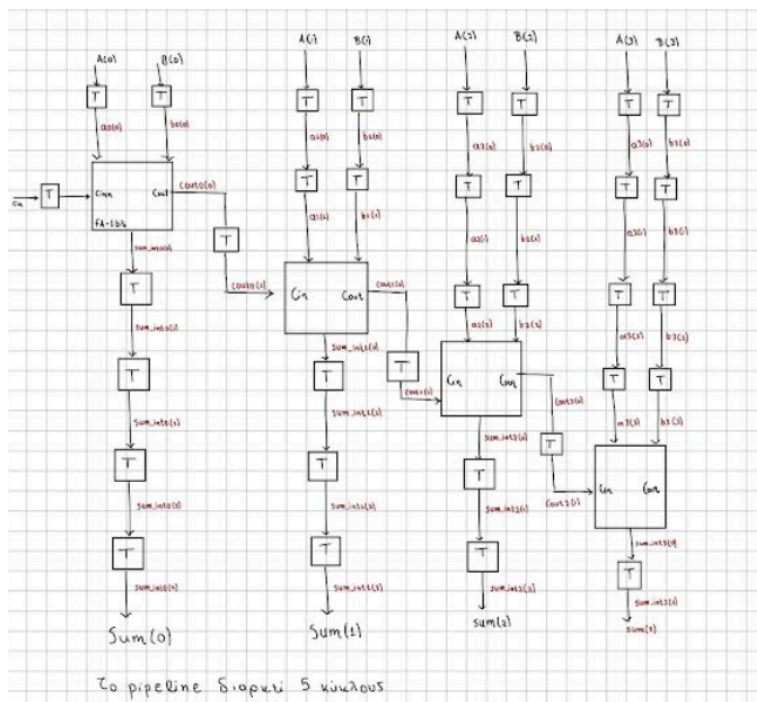
Τέλος γνωρίζουμε ότι το critical path είναι το path με την μεγαλύτερη συνολική καθυστέρηση

Name	Slack	Levels	Routes	High Fanout	From	To	Total Delay	Logic Delay	Net Delay	Requirement	Source Clock
Path 1	∞	3	2	2	A	Cout	7.030	3.939	3.091	∞	input port clock
Path 2	∞	3	2	2	A	Sum	6.938	3.717	3.221	∞	input port clock

Στη συγκεκριμένη περίπτωση παρατηρούμε ότι είναι το path από την είσοδο A στο Cout.

2. Υλοποίηση ενός σύγχρονου Αθροιστή διάδοσης κρατουμένου των 4 bits με χρήση της τεχνικής Pipeline.

Το κύκλωμα θα πρέπει να τροφοδοτείται με ένα διαφορετικό ζεύγος εισόδων σε κάθε κύκλο ρολογιού και να δίνει αντίστοιχα ορθό αποτέλεσμα σε κάθε κύκλο ρολογιού έπειτα από κάποια αρχική καθυστέρηση Latency . Στην παρούσα άσκηση υλοποιήσαμε το εξής κύκλωμα:



Περιμένουμε το Latency να είναι 5 κύκλοι του ρολογιού αφού το pipeline διαρκεί 5 στάδια, επομένως μέχρι να ολοκληρωθεί η πρώτη πρόσθεση θα χρειαστούν 5 κύκλοι. Τοποθετούμε σε κάθε στάδιο registers ώστε τα δεδομένα να πηγαίνουν στην έξοδο με τον σωστό χρονισμό.

Ο κώδικας της άσκησης:

```
library IEEE;  
use IEEE.STD_LOGIC_1164.ALL;
```

```
entity pipelined_full_adder is  
  Port (A:in std_logic_vector(3 downto 0);  
        B:in std_logic_vector(3 downto 0);  
        Cin:in std_logic;  
        Cout:out std_logic;  
        clk:in std_logic;  
        sum:out std_logic_vector(3 downto 0));  
  
end pipelined_full_adder;  
  
architecture Behavioral of pipelined_full_adder is  
  
  component D_flip_flop is  
    Port ( D : in STD_LOGIC;  
          CLK : in STD_LOGIC;  
          RSTn : in STD_LOGIC;  
          Q : out STD_LOGIC);  
  end component;  
  
  component full_adder1 is  
    Port ( A : in STD_LOGIC;  
          B : in STD_LOGIC;  
          Cin : in STD_LOGIC;  
          Cout : out STD_LOGIC;  
          Sum : out STD_LOGIC);  
  end component;  
  
  signal a0:std_logic;  
  signal b0:std_logic;  
  signal a1:std_logic_vector(1 downto 0);  
  signal b1:std_logic_vector(1 downto 0);  
  signal a2:std_logic_vector(2 downto 0);  
  signal b2:std_logic_vector(2 downto 0);  
  signal a3:std_logic_vector(3 downto 0);  
  signal b3:std_logic_vector(3 downto 0);  
  signal cinn:std_logic;  
  signal sum_int0:std_logic_vector(4 downto 0);  
  signal sum_int1:std_logic_vector(3 downto 0);  
  signal sum_int2:std_logic_vector(2 downto 0);  
  signal sum_int3:std_logic;
```

```
signal cout0:std_logic_vector(2 downto 0);
signal cout1:std_logic_vector(2 downto 0);
signal cout2:std_logic_vector(2 downto 0);
```

```
begin
```

```
--delay of A0
```

```
dffa0: d_flip_flop port map (A(0),clk,'0',a0); --done
```

```
--delay of b0
```

```
dff1b0: d_flip_flop port map (B(0),clk,'0',b0);--done
```

```
dff1b9: d_flip_flop port map (cin,clk,'0',cinn);--done
```

```
--delay of cin
```

```
FA1: full_adder1 port map (a0,b0, cinn,cout0(0),sum_int0(0));--done
```

```
--delay of sum0
```

```
dff1: d_flip_flop port map (sum_int0(0),clk,'0',sum_int0(1));--done
```

```
--1cc
```

```
dff2: d_flip_flop port map (sum_int0(1),clk,'0',sum_int0(2));--done
```

```
--2cc
```

```
dff3: d_flip_flop port map (sum_int0(2),clk,'0',sum_int0(3));--done
```

```
--3cc
```

```
dff44: d_flip_flop port map (sum_int0(3),clk,'0',sum_int0(4));--done
```

```
sum(0)<=sum_int0(4);
```

```
--delay of A1
```

```
dffa10: d_flip_flop port map (A(1),clk,'0',a1(0));--done
```

```
--1cc
```

```
dffa11: d_flip_flop port map (a1(0),clk,'0',a1(1));--done
```

```
--2cc
```

```
--delay of B1
```

```
dffb10: d_flip_flop port map(B(1),clk,'0',b1(0));--done
```

```
--1cc
```

```
dffb11: d_flip_flop port map (b1(0),clk,'0',b1(1));--done
```

```
--2cc
```

```
--delay of cout0
```

```
dff6: d_flip_flop port map (cout0(0),clk,'0', cout0(1));--done
```

```
--dff66: d_flip_flop port map (cout0(1),'0',clk, cout0(2));--done
```

```
FA2: full_adder1 port map (a1(1),b1(1),cout0(1),cout1(0),sum_int1(0));--done
```

```
--delay of sum1
```

```
dff7: d_flip_flop port map (sum_int1(0),clk,'0',sum_int1(1));--done
```

```
--1cc
```

```
dff9: d_flip_flop port map (sum_int1(1),clk,'0',sum_int1(2));--done
```

```
--2cc
```

```
dff99: d_flip_flop port map (sum_int1(2),clk,'0',sum_int1(3));--done
```

```
sum(1)<=sum_int1(3);
```

```
--delay of A2
```

```
dffaa20: d_flip_flop port map (A(2),clk,'0',a2(0));--done
```

```
--1cc
```

```
dff1a21: d_flip_flop port map (a2(0),clk,'0',a2(1));--done
```

```

--2cc
dffa22: d_flip_flop port map (a2(1),clk,'0',a2(2)); --done
--delay of B2
dff11: d_flip_flop port map (B(2),clk,'0',b2(0));--done
--1cc
dff13: d_flip_flop port map (b2(0),clk,'0',b2(1));--done
--2cc
dff12: d_flip_flop port map (b2(1),clk,'0',b2(2));--done
--delay of cout1
dff14: d_flip_flop port map (cout1(0),clk,'0', cout1(1));--done

--dff144: d_flip_flop port map (cout1(1),'0',clk, cout1(2));--done
--1cc
FA3: full_adder1 port map (a2(2),b2(2),cout1(1),cout2(0),sum_int2(0));--done
--delay of sum2
dff15: d_flip_flop port map (sum_int2(0),clk,'0', sum_int2(1));--done
--1cc
dff155: d_flip_flop port map (sum_int2(1),clk,'0', sum_int2(2));--done
sum(2)<=sum_int2(2);
--lets go to the final stage
--delay of A3
dffa30: d_flip_flop port map (A(3),clk,'0', a3(0));--done
--1cc
dffa31: d_flip_flop port map (a3(0),clk,'0', a3(1));--done
--2cc
dffa32: d_flip_flop port map (a3(1),clk,'0', a3(2));--done
--3cc
dffa33: d_flip_flop port map (a3(2),clk,'0', a3(3));--done

--delay of B3
dffb30: d_flip_flop port map (B(3),clk,'0', b3(0));--done
--1cc
dffb31: d_flip_flop port map (b3(0),clk,'0', b3(1));--done
--2cc
dffb32: d_flip_flop port map (b3(1),clk,'0', b3(2));--done
--3cc
dffb33: d_flip_flop port map (b3(2),clk,'0', b3(3));--done
--delay of cout1
dff22: d_flip_flop port map (cout2(0),clk,'0', cout2(1));--done
--1cc
--dff222: d_flip_flop port map (cout2(1),'0',clk, cout2(2));--done

FA4: full_adder1 port map (b3(3),a3(3),cout2(1),cout,sum_int3);--done

dffb304: d_flip_flop port map (sum_int3,clk,'0', sum(3));--done

end Behavioral;

```

KQ1 TO test bench:

```
library IEEE;  
use IEEE.STD_LOGIC_1164.ALL;
```

entity test is

end test;

architecture Behavioral of test is

```
component pipelined_full_adder is  
  Port ( A : in std_logic_vector(3 downto 0);  
        B : in std_logic_vector(3 downto 0);  
        Cin : in std_logic;  
        Cout: out std_logic;  
        clk : in std_logic;  
        sum : out std_logic_vector(3 downto 0));  
end component;
```

```
signal A_tb : std_logic_vector(3 downto 0);  
signal B_tb : std_logic_vector(3 downto 0);  
signal Cin_tb : std_logic;  
signal Cout_tb : std_logic;  
signal clk_tb : std_logic;  
signal sum_tb : std_logic_vector(3 downto 0);
```

begin

```
DUT: pipelined_full_adder port map (  
  A => A_tb,  
  B => B_tb,  
  Cin => Cin_tb,  
  Cout => Cout_tb,  
  clk => clk_tb,  
  sum => sum_tb  
);
```

```
clk_process : process  
begin  
  clk_tb <= '0';  
  wait for 5 ns;  
  clk_tb <= '1';  
  wait for 5 ns;  
end process;
```

stimulus: process

```

begin
  wait for 10ns;

  A_tb <= "0000";
  B_tb <= "0000";
  Cin_tb <= '0';
  wait for 20 ns;

  A_tb <= "0101";
  B_tb <= "1010";
  Cin_tb <= '0';
  wait for 20 ns;

  A_tb <= "1111";
  B_tb <= "0000";
  Cin_tb <= '0';
  wait for 20 ns;

  A_tb <= "1110";
  B_tb <= "1110";
  Cin_tb <= '0';
  wait for 20 ns;

  A_tb <= "1000";
  B_tb <= "1110";
  Cin_tb <= '0';
  wait for 20 ns;

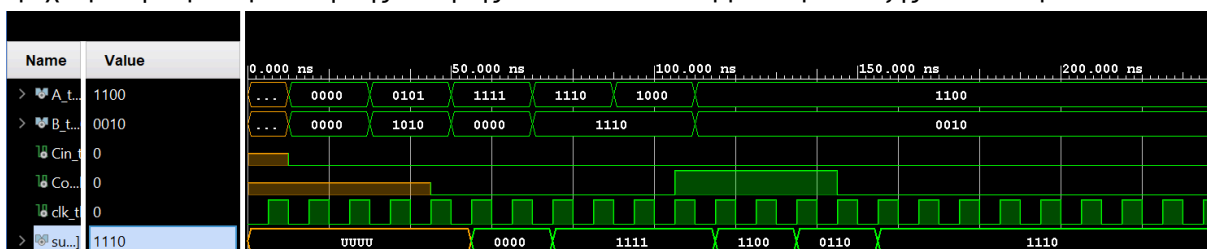
  A_tb <= "1100";
  B_tb <= "0010";
  Cin_tb <= '0';
  wait for 20 ns;

  wait;
end process;

end Behavioral;

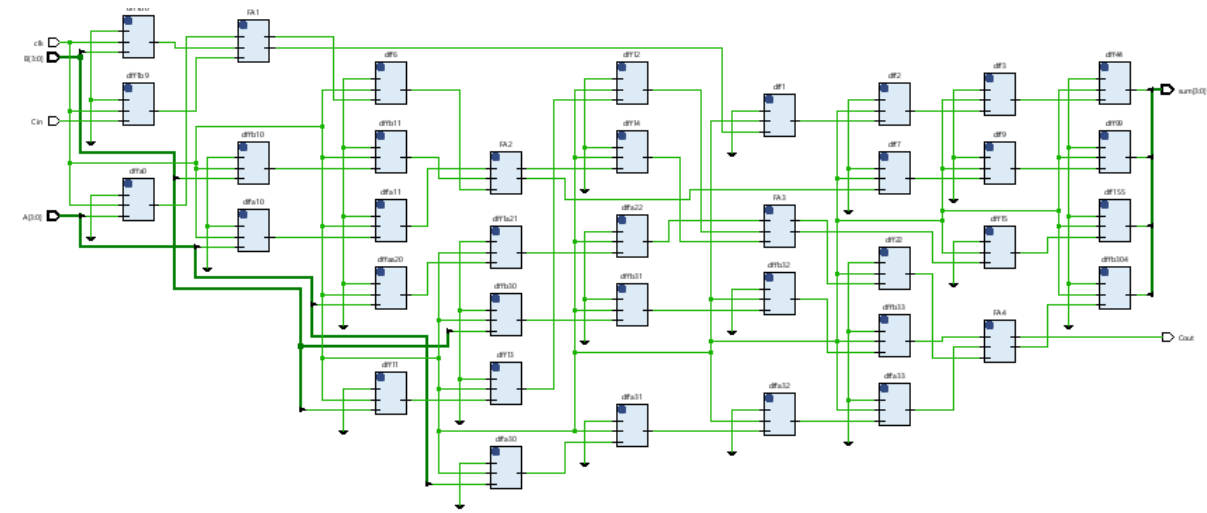
```

Τρέχουμε την προσομοίωση της άσκησης στο vivado και λαμβάνουμε το εξής αποτέλεσμα:



Παρατηρούμε ότι το πρόγραμμά μας λειτουργεί ως πλήρης αθροιστής με pipeline και αρχική καθυστέρηση.

Κατόπιν, ανοίγουμε το RTL schematic:



και τέλος με τη σύνθεση προσδιορίζουμε το critical path:

Name	Slack	Levels	High Fanout	From	To	Total Delay	Logic Delay	Net Delay	Requirement
Path 1	∞	3	2	dffa33/q_reg/C	Cout	6.381	3.398	2.983	∞
Path 2	∞	2	1	dffb99/q_reg/C	sum[1]	5.118	3.260	1.858	∞
Path 3	∞	2	1	dffb44/q_reg/C	sum[0]	4.972	3.256	1.716	∞
Path 4	∞	2	1	dffb304/q_reg/C	sum[3]	4.916	3.055	1.861	∞
Path 5	∞	2	1	dffb155/q_reg/C	sum[2]	4.778	3.113	1.665	∞
Path 6	∞	2	2	dffb1b9/q req/C	dffb3/q req srl3/D	2.003	0.580	1.423	∞

Όπως διαπιστώνουμε το critical path είναι το path 1 με συνολική καθυστέρηση 6.381ns.

3. Υλοποίηση ενός συστολικού (είδος pipeline) Πολλαπλασιαστή διάδοσης κρατουμένων των 4 bits κάνοντας χρήση σύγχρονων Πλήρων Αθροιστών (Full Adders).

Το κύκλωμα τροφοδοτείται με ένα διαφορετικό ζεύγος εισόδων σε κάθε κύκλο ρολογιού και δίνει αντίστοιχα ορθό αποτέλεσμα σε κάθε κύκλο ρολογιού έπειτα από κάποια αρχική καθυστέρηση Latency .

Το κύκλωμα του **pipelined πολλαπλασιαστή** που υλοποιήσαμε:

1. Αρχικά παραθέτουμε τον κώδικα του black box που δημιουργήσαμε για να διευκολυνθεί η υλοποίηση του πολλαπλασιαστή.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
```

entity blackbox is

Port (Abb:in std_logic_vector(3 downto 0);

```

        Bbb:in std_logic;
        Cinbb:in std_logic_vector(3 downto 0);
        inFA: in std_logic_vector(3 downto 0);
        Coutbb:out std_logic_vector(3 downto 0);
        factor:out std_logic_vector(3 downto 0));
end blackbox;

```

architecture Behavioral of blackbox is

```

component full_adder1
  Port ( A: in std_logic;
        B: in std_logic;
        Cin: in std_logic;
        Sum: out std_logic;
        Cout: out std_logic);
end component;

```

```

signal andd:std_logic_vector(3 downto 0);

```

```

begin
  andd(0)<= Abb(0) and Bbb;
  andd(1)<= Abb(1) and Bbb;
  andd(2)<= Abb(2) and Bbb;
  andd(3)<= Abb(3) and Bbb;
  FA1: full_adder1 port map(andd(0),inFA(0),Cinbb(0),factor(0),coutbb(0));
  FA2: full_adder1 port map(andd(1),inFA(1),Cinbb(1),factor(1),coutbb(1));
  FA3: full_adder1 port map(andd(2),inFA(2),Cinbb(2),factor(2),coutbb(2));
  FA4: full_adder1 port map(andd(3),inFA(3),Cinbb(3),factor(3),coutbb(3));

```

```

end Behavioral;

```

Ο ολοκληρωμένος κώδικας του πολλαπλασιαστή _____ :

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

```

```

entity pipelined_multiplier is
  Port (Ain: in std_logic_vector(3 downto 0);
        Bin: in std_logic_vector(3 downto 0);
        Mul: out std_logic_vector(7 downto 0);
        Cout:out std_logic;
        clk: in std_logic
  );
end pipelined_multiplier;

```

architecture Behavioral of pipelined_multiplier is

component d_flip_flop is

```

  Port (d : in std_logic;
        clk : in std_logic;
        RSTn : in std_logic;

```

```
    q : out std_logic);  
end component;
```

component blackbox is

```
    Port ( Abb:in std_logic_vector(3 downto 0);  
          Bbb:in std_logic;  
          Cinbb:in std_logic_vector(3 downto 0);  
          inFA: in std_logic_vector(3 downto 0);  
          Coutbb:out std_logic_vector(3 downto 0);  
          factor:out std_logic_vector(3 downto 0));  
end component;
```

component full_adder1 is

```
    Port ( A, B, Cin : in  STD_LOGIC;  
          Sum, Cout : out STD_LOGIC);  
end component;
```

```
signal a:std_logic_vector(3 downto 0);  
signal b_0:std_logic;  
signal b_1:std_logic_vector(1 downto 0);  
signal b_2:std_logic_vector(2 downto 0);  
signal b_3:std_logic_vector(3 downto 0);  
signal cout0: std_logic_vector(3 downto 0);  
signal cout1:std_logic_vector(3 downto 0);  
signal cout2:std_logic_vector(3 downto 0);  
signal cout3:std_logic_vector(3 downto 0);  
signal cout4:std_logic_vector(3 downto 0);  
signal cout00:std_logic_vector(3 downto 0);  
signal cout11:std_logic_vector(3 downto 0);  
signal cout22:std_logic_vector(3 downto 0);  
signal cout33:std_logic_vector(3 downto 0);  
signal factor0:std_logic_vector(3 downto 0);  
signal factor1:std_logic_vector(3 downto 0);  
signal factor2:std_logic_vector(3 downto 0);  
signal factor3:std_logic_vector(3 downto 0);  
signal factor4:std_logic_vector(3 downto 0);  
signal factor0_copy:std_logic_vector(3 downto 0);  
signal factor1_copy:std_logic_vector(3 downto 0);  
signal factor2_copy:std_logic_vector(3 downto 0);  
signal factor3_copy:std_logic_vector(3 downto 0);  
signal factor0_copy_shifted:std_logic_vector(3 downto 0);  
signal factor1_copy_shifted:std_logic_vector(3 downto 0);  
signal factor2_copy_shifted:std_logic_vector(3 downto 0);  
signal factor3_copy_shifted:std_logic_vector(3 downto 0);  
signal a1:std_logic_vector(3 downto 0);  
signal a11:std_logic_vector(3 downto 0);  
signal a2:std_logic_vector(3 downto 0);  
signal a22:std_logic_vector(3 downto 0);  
signal a23:std_logic_vector(3 downto 0);  
signal a3:std_logic_vector(3 downto 0);  
signal a32:std_logic_vector(3 downto 0);  
signal a33:std_logic_vector(3 downto 0);
```

```

signal a34:std_logic_vector(3 downto 0);
signal mul_0:std_logic_vector(4 downto 0);
signal mul_1:std_logic_vector(3 downto 0);
signal mul_2:std_logic_vector(2 downto 0);
signal mul_3:std_logic_vector(1 downto 0);
signal c1:std_logic_vector(3 downto 0);
signal c2:std_logic_vector(3 downto 0);
signal c3:std_logic_vector(3 downto 0);
signal c4:std_logic;
signal c5:std_logic_vector(4 downto 0);
signal c6:std_logic;
signal c7:std_logic;
signal c8:std_logic;
signal c9:std_logic_vector(1 downto 0);

```

```

begin
-- 1cc delay of A0(0)
DFF_a_0: d_flip_flop port map (Ain(0),clk,'0',a(0));
-- 1cc delay of A1(0)
DFF_a_1: d_flip_flop port map (Ain(1),clk,'0',a(1));
-- 1cc delay of A2(0)
DFF_a_2: d_flip_flop port map (Ain(2),clk,'0',a(2));
-- 1cc delay of A3(0)
DFF_a_3: d_flip_flop port map (Ain(3),clk,'0',a(3));
-- 1cc delay of B0(0)
DFF_b_0: d_flip_flop port map (Bin(0),clk,'0',b_0);
c1<="0000";
DFF_b_000: d_flip_flop port map (c1(0),clk,'0',c2(0));
DFF_b_001: d_flip_flop port map (c1(1),clk,'0',c2(1));
DFF_b_010: d_flip_flop port map (c1(2),clk,'0',c2(2));
DFF_b_100: d_flip_flop port map (c1(3),clk,'0',c2(3));

```

```

--now the first black box
black_box_1: blackbox port map(a,b_0,c2,c2,cout0,factor0);--factor0(0)

```

```

-- 2cc delay of A0
DFF_a1_0: d_flip_flop port map (Ain(0),clk,'0',a1(0));
DFF_a11_0: d_flip_flop port map (a1(0),clk,'0',a11(0));
-- 2cc delay of A1
DFF_a1_1: d_flip_flop port map (Ain(1),clk,'0',a1(1));
DFF_a11_1: d_flip_flop port map (a1(1),clk,'0',a11(1));
-- 2cc delay of A2
DFF_a1_2: d_flip_flop port map (Ain(2),clk,'0',a1(2));
DFF_a11_2: d_flip_flop port map (a1(2),clk,'0',a11(2));
-- 2cc delay of A3
DFF_a1_3: d_flip_flop port map (Ain(3),clk,'0',a1(3));
DFF_a11_3: d_flip_flop port map (a1(3),clk,'0',a11(3));
--2cc delay of b_1

```

```

DFF_b_1: d_flip_flop port map(Bin(1),clk,'0',b_1(0));
DFF_b_12: d_flip_flop port map(b_1(0),clk,'0',b_1(1));
--delay of cin0(0)
DFF_cout0: d_flip_flop port map(Cout0(0),clk,'0',cout00(0));
--delay of cin0(1)
DFF_cout01: d_flip_flop port map(Cout0(1),clk,'0',cout00(1));
--delay of cin0(2)
DFF_cout02: d_flip_flop port map(Cout0(2),clk,'0',cout00(2));
--delay of cin0(3)
DFF_cout03: d_flip_flop port map(Cout0(3),clk,'0',cout00(3));
--delay of factor0_0
DFF_factor0_0: d_flip_flop port map(factor0(0),clk,'0',factor0_copy(0));--this for the exit
--delay of factor0_1
DFF_factor0_1: d_flip_flop port map(factor0(1),clk,'0',factor0_copy(1));
--delay of factor0_2
DFF_factor0_2: d_flip_flop port map(factor0(2),clk,'0',factor0_copy(2));
--delay of factor0_3
DFF_factor0_3: d_flip_flop port map(factor0(3),clk,'0',factor0_copy(3));
--second black box
factor0_copy_shifted <= "0" & factor0_copy(3 downto 1);
black_box_2: blackbox port map(a11,b_1(1),cout00,factor0_copy_shifted,cout1,factor1);

```

--lets go to the third stage

--3cc delay of A0

```

DFF_a2_0: d_flip_flop port map (Ain(0),clk,'0',a2(0));
DFF_a22_0: d_flip_flop port map (a2(0),clk,'0',a22(0));
DFF_a23_0: d_flip_flop port map (a22(0),clk,'0',a23(0));

```

--3cc delay of A1

```

DFF_a2_1: d_flip_flop port map (Ain(1),clk,'0',a2(1));
DFF_a22_1: d_flip_flop port map (a2(1),clk,'0',a22(1));
DFF_a23_1: d_flip_flop port map (a22(1),clk,'0',a23(1));

```

--3cc delay of A2

```

DFF_a2_2: d_flip_flop port map (Ain(2),clk,'0',a2(2));
DFF_a22_2: d_flip_flop port map (a2(2),clk,'0',a22(2));
DFF_a23_2: d_flip_flop port map (a22(2),clk,'0',a23(2));

```

--3cc delay of A3

```

DFF_a2_3: d_flip_flop port map (Ain(3),clk,'0',a2(3));
DFF_a22_3: d_flip_flop port map (a2(3),clk,'0',a22(3));
DFF_a23_3: d_flip_flop port map (a22(3),clk,'0',a23(3));

```

--3cc delay of b2

```

DFF_b_2: d_flip_flop port map(Bin(2),clk,'0',b_2(0));
DFF_b_22: d_flip_flop port map(b_2(0),clk,'0',b_2(1));
DFF_b_23: d_flip_flop port map(b_2(1),clk,'0',b_2(2));

```

--delay of cin1(0)

```

DFF_cout10: d_flip_flop port map(Cout1(0),clk,'0',cout11(0));
--delay of cin1(1)

```

```

DFF_cout11: d_flip_flop port map(Cout1(1),clk,'0',cout11(1));
--delay of cin1(2)

```

```

DFF_cout12: d_flip_flop port map(Cout1(2),clk,'0',cout11(2));
--delay of cin1(3)

```

```

DFF_cout13: d_flip_flop port map(Cout1(3),clk,'0',cout11(3));
--delay of factor1_0

```

```

DFF_factor1_0: d_flip_flop port map(factor1(0),clk,'0',factor1_copy(0));--this for the exit
--delay of factor0_1
DFF_factor1_1: d_flip_flop port map(factor1(1),clk,'0',factor1_copy(1));
--delay of factor0_2
DFF_factor1_2: d_flip_flop port map(factor1(2),clk,'0',factor1_copy(2));
--delay of factor0_3
DFF_factor1_3: d_flip_flop port map(factor1(3),clk,'0',factor1_copy(3));
--lets go to the third black box
factor1_copy_shifted<= "0" & factor1_copy(3 downto 1);
black_box_3: blackbox port map(a23,b_2(2),cout11,factor1_copy_shifted,cout2,factor2);

```

--now the fourth stage

```

--4cc delay of A0
DFF_a3_0: d_flip_flop port map (Ain(0),clk,'0',a3(0));
DFF_a32_0: d_flip_flop port map (a3(0),clk,'0',a32(0));
DFF_a33_0: d_flip_flop port map (a32(0),clk,'0',a33(0));
DFF_a34_0: d_flip_flop port map (a33(0),clk,'0',a34(0));
--4cc delay of A1
DFF_a3_1: d_flip_flop port map (Ain(1),clk,'0',a3(1));
DFF_a32_1: d_flip_flop port map (a3(1),clk,'0',a32(1));
DFF_a33_1: d_flip_flop port map (a32(1),clk,'0',a33(1));
DFF_a34_1: d_flip_flop port map (a33(1),clk,'0',a34(1));
--4cc delay of A2
DFF_a3_2: d_flip_flop port map (Ain(2),clk,'0',a3(2));
DFF_a32_2: d_flip_flop port map (a3(2),clk,'0',a32(2));
DFF_a33_2: d_flip_flop port map (a32(2),clk,'0',a33(2));
DFF_a34_2: d_flip_flop port map (a33(2),clk,'0',a34(2));
--4cc delay of A3
DFF_a3_3: d_flip_flop port map (Ain(3),clk,'0',a3(3));
DFF_a32_3: d_flip_flop port map (a3(3),clk,'0',a32(3));
DFF_a33_3: d_flip_flop port map (a32(3),clk,'0',a33(3));
DFF_a34_3: d_flip_flop port map (a33(3),clk,'0',a34(3));
--4cc delay of B3
DFF_b_3: d_flip_flop port map(Bin(3),clk,'0',b_3(0));
DFF_b_32: d_flip_flop port map(b_3(0),clk,'0',b_3(1));
DFF_b_33: d_flip_flop port map(b_3(1),clk,'0',b_3(2));
DFF_b_34: d_flip_flop port map(b_3(2),clk,'0',b_3(3));
--delay of cin2(0)
DFF_cout20: d_flip_flop port map(Cout2(0),clk,'0',cout22(0));
--delay of cin2(1)
DFF_cout21: d_flip_flop port map(Cout2(1),clk,'0',cout22(1));
--delay of cin2(2)
DFF_cout22: d_flip_flop port map(Cout2(2),clk,'0',cout22(2));
--delay of cin1(3)
DFF_cout23: d_flip_flop port map(Cout2(3),clk,'0',cout22(3));
--delay of factor2_0
DFF_factor2_0: d_flip_flop port map(factor2(0),clk,'0',factor2_copy(0));--this for the exit
--delay of factor0_1
DFF_factor2_1: d_flip_flop port map(factor2(1),clk,'0',factor2_copy(1));
--delay of factor0_2

```

```

DFF_factor2_2: d_flip_flop port map(factor2(2),clk,'0',factor2_copy(2));
--delay of factor0_3
DFF_factor2_3: d_flip_flop port map(factor2(3),clk,'0',factor2_copy(3));
factor2_copy_shifted <= "0" & factor2_copy(3 downto 1);
--lets go to the fourth black box
black_box_4: blackbox port map(a34,b_3(3),cout22,factor2_copy_shifted,cout3,factor3);

--delay of cin3(0)
DFF_cout30: d_flip_flop port map(Cout3(0),clk,'0',cout33(0));
--delay of cin3(1)
DFF_cout31: d_flip_flop port map(Cout3(1),clk,'0',cout33(1));
--delay of cin3(2)
DFF_cout32: d_flip_flop port map(Cout3(2),clk,'0',cout33(2));
--delay of cin3(3)
DFF_cout33: d_flip_flop port map(Cout3(3),clk,'0',cout33(3));

DFF_factor3_0: d_flip_flop port map(factor3(0),clk,'0',factor3_copy(0));--this for the exit
--delay of factor3_1
DFF_factor3_1: d_flip_flop port map(factor3(1),clk,'0',factor3_copy(1));
--delay of factor3_2
DFF_factor3_2: d_flip_flop port map(factor3(2),clk,'0',factor3_copy(2));
--delay of factor3_3
DFF_factor3_3: d_flip_flop port map(factor3(3),clk,'0',factor3_copy(3));
factor3_copy_shifted <= "0" & factor3_copy(3 downto 1);
DFF22: d_flip_flop port map('0',clk,'0',c5(0));
DFF222: d_flip_flop port map(c5(0),clk,'0',c5(1));
DFF2222: d_flip_flop port map(c5(1),clk,'0',c5(2));
DFF223: d_flip_flop port map(c5(2),clk,'0',c5(3));
DFF225: d_flip_flop port map(c5(3),clk,'0',c5(4));

FA:full_adder1 port map(cout33(0),factor3_copy(1),c5(4),factor4(0),cout4(0));
--FA5:full_adder1 port map('0','0',c5(4),factor4(0),cout4(0));
FA1:full_adder1 port map(cout33(1),factor3_copy_shifted(1),cout4(0),factor4(1),cout4(1));
FA2:full_adder1 port map(cout33(2),factor3_copy_shifted(2),cout4(1),factor4(2),cout4(2));
FA3:full_adder1 port map(cout33(3),factor3_copy_shifted(3),cout4(2),factor4(3),cout4(3));

dffmul0: d_flip_flop port map(factor0(0),clk,'0',mul_0(0));
dffmul01: d_flip_flop port map(mul_0(0),clk,'0',mul_0(1));
dffmul011: d_flip_flop port map(mul_0(1),clk,'0',mul_0(2));
dffmul0111: d_flip_flop port map(mul_0(2),clk,'0',mul_0(3));
dffmul01111: d_flip_flop port map(mul_0(3),clk,'0',mul_0(4));
mul(0)<=mul_0(4);
dffmul1: d_flip_flop port map(factor1(0),clk,'0',mul_1(0));
dffmul11: d_flip_flop port map(mul_1(0),clk,'0',mul_1(1));
dffmul001111: d_flip_flop port map(mul_1(1),clk,'0',mul_1(2));
dffmul0011311: d_flip_flop port map(mul_1(2),clk,'0',mul_1(3));

mul(1)<=mul_1(3);
dffmul02: d_flip_flop port map(factor2(0),clk,'0',mul_2(0));
dffmul2201: d_flip_flop port map(mul_2(0),clk,'0',mul_2(1));
dffmul223301: d_flip_flop port map(mul_2(1),clk,'0',mul_2(2));

```

```

mul(2)<=mul_2(2);
dffmul40: d_flip_flop port map(factor3(0),clk,'0',mul_3(0));
dffmul22501: d_flip_flop port map(mul_3(0),clk,'0',mul_3(1));
mul(3)<=mul_3(1);
dffmul22601: d_flip_flop port map(factor4(0),clk,'0',c4);
mul(4)<=c4;
dffmul1: d_flip_flop port map(factor4(1),clk,'0',c6);
mul(5)<=c6;
dffmua: d_flip_flop port map(factor4(2),clk,'0',c7);
mul(6)<=c7;
dffmulb: d_flip_flop port map(factor4(3),clk,'0',c8);
mul(7)<=c8;
DFF224: d_flip_flop port map(cout33(0),clk,'0',c9(0));
cout<=c9(0);

```

To test bench

Ελέγχουμε την ορθή λειτουργία του κυκλώματος με κάποιες παραδειγματικές τιμές.

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

```

```

entity pipelined_multiplier_tb is
end pipelined_multiplier_tb;

```

architecture behavior of pipelined_multiplier_tb is

```

    signal Ain      : std_logic_vector(3 downto 0);
    signal Bin      : std_logic_vector(3 downto 0);
    signal Mul      : std_logic_vector(7 downto 0);
    signal Cout     : std_logic;
    signal clk      : std_logic;

```

```
begin
```

```

    uut: entity work.pipelined_multiplier
        port map (
            Ain => Ain,
            Bin => Bin,
            Mul => Mul,
            Cout => Cout,
            clk => clk
        );

```

```

    clk_process: process
    begin
        clk <= '0';
        wait for 10 ns;

```

```
    clk <= '1';  
    wait for 10 ns;  
end process;
```

```
stimulus_process: process  
begin
```

```
    Ain <= "0000";  
    Bin <= "0000";  
    wait for 20 ns;
```

```
    Ain <= "0010";  
    Bin <= "0011";  
    wait for 40 ns;
```

```
    Ain <= "0101";  
    Bin <= "0111";  
    wait for 40 ns;
```

```
    Ain <= "1111";  
    Bin <= "1111";  
    wait for 40 ns;
```

```
    Ain <= "0100";  
    Bin <= "0100";  
    wait for 40 ns;
```

```
    Ain <= "0111";  
    Bin <= "0111";  
    wait for 40 ns;
```

```
    Ain <= "1111";  
    Bin <= "0111";  
    wait for 40 ns;
```

```
    Ain <= "0101";  
    Bin <= "0100";  
    wait for 40 ns;
```

```
    Ain <= "0110";  
    Bin <= "0010";  
    wait for 40 ns;
```

```
    Ain <= "1110";  
    Bin <= "0010";  
    wait for 40 ns;
```

```
Ain <= "1000";  
Bin <= "0010";  
wait for 40 ns;
```

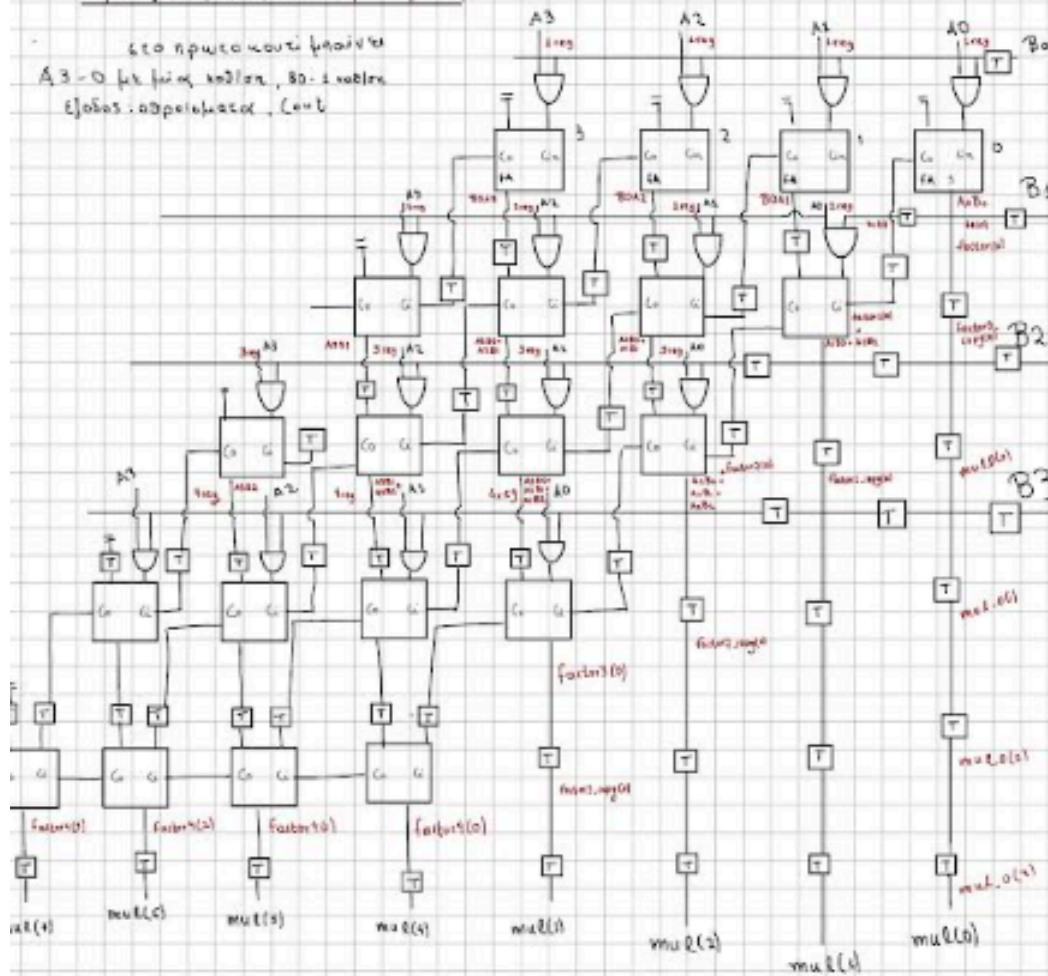
```
Ain <= "1000";  
Bin <= "1000";
```

```
wait for 40ns;  
wait;  
end process;
```

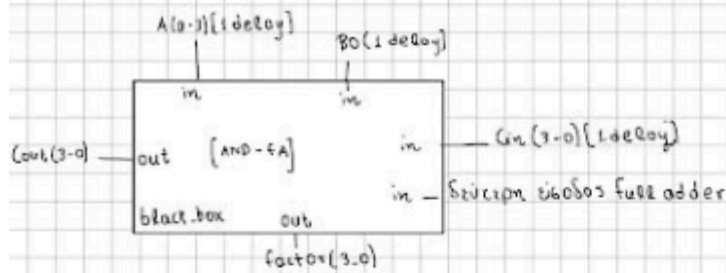
Unsigned Multiplication

	A_1	A_2	A_3	A_0
$\times B_3$	B_3	B_2	B_1	B_0
	$A_1 B_0$	$A_2 B_0$	$A_3 B_0$	$A_0 B_0$
+	$A_1 B_1$	$A_2 B_1$	$A_3 B_1$	$A_0 B_1$
	$A_1 B_2$	$A_2 B_2$	$A_3 B_2$	$A_0 B_2$
	$A_1 B_3$	$A_2 B_3$	$A_3 B_3$	$A_0 B_3$

6 to 8 pins 4046 provided
 A3-0 for pin 10/12, 80-1 values
 0/000: address, 1000: 1000

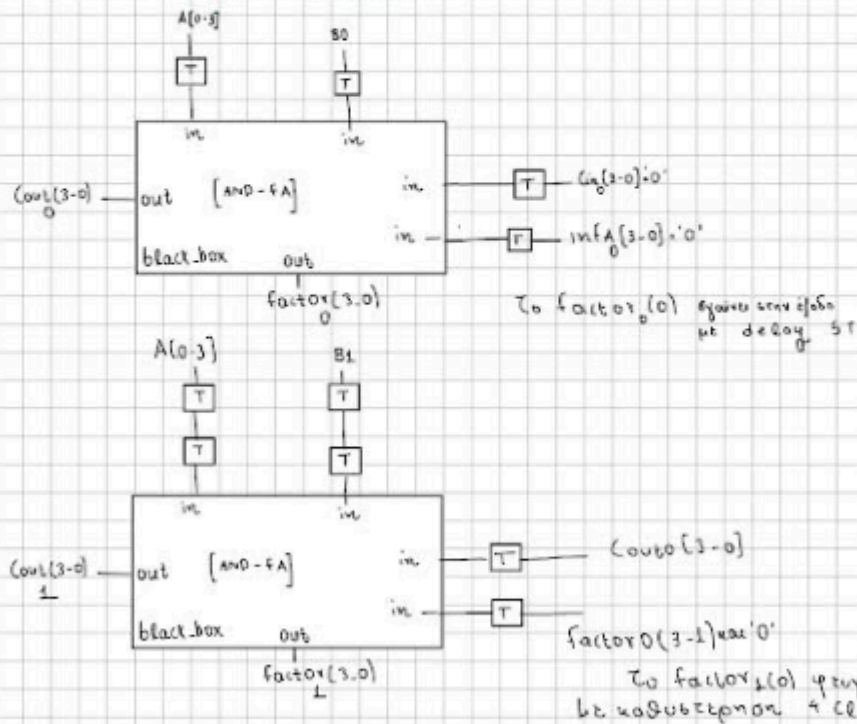


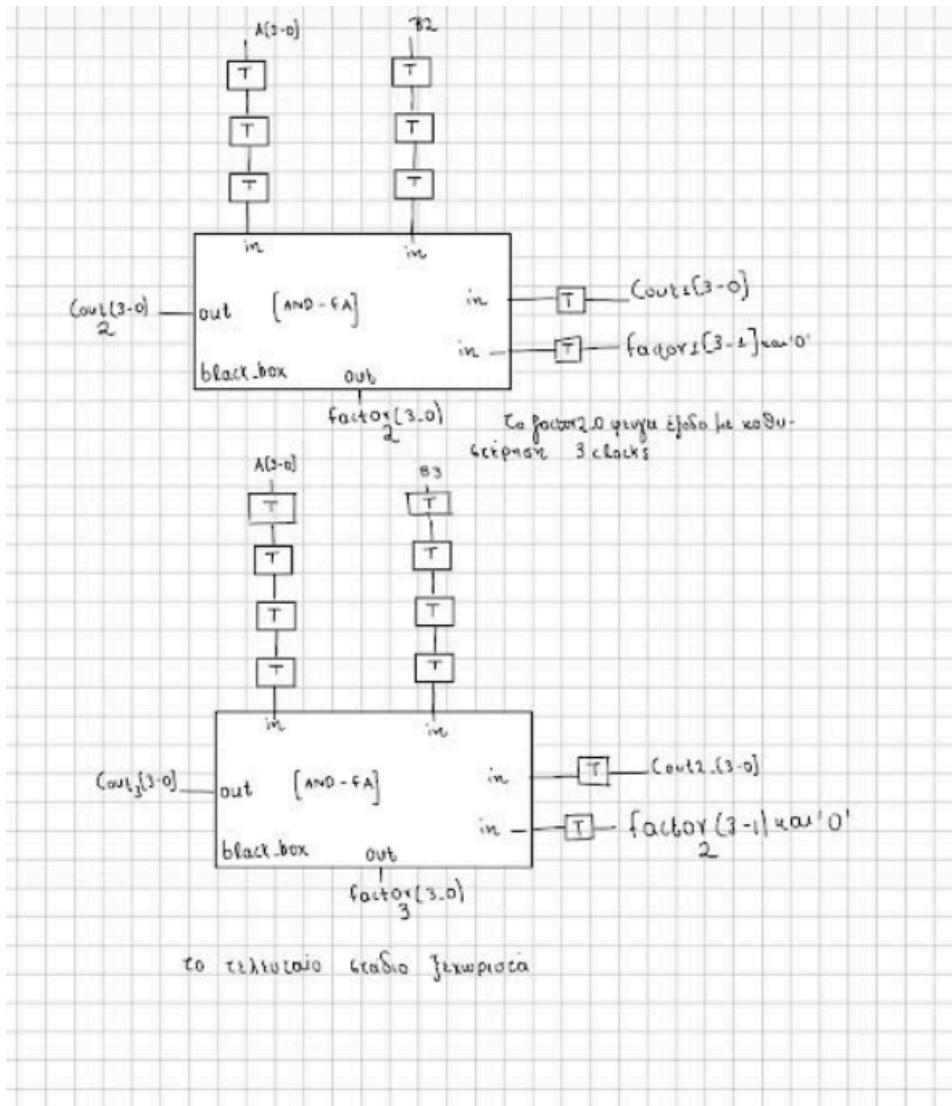
Θα σχεδιάσω πρώτα 4 μαύρα κουτιά



το ονομάζω black box και το χρησιμοποιώ στην τελική υλοποίηση

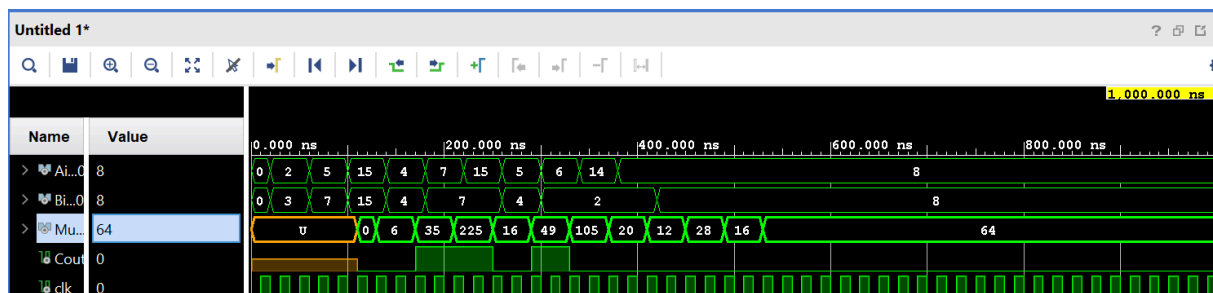
Συνολικό Κυκλώμα Multiplier





Για να φτιάξουμε τον πολλαπλασιαστή χρησιμοποιούμε το κυκλωμα της πρώτης εικόνας. Για το σκοπό αυτό δημιουργούμε μια συνδυαστική δομή η οποία περιλαμβάνει τέσσερις full adders και τέσσερις πύλες and. Τοποθετούμε registers ανάμεσα σε κάθε μαύρο κουτί με τέτοιο τρόπο ώστε τα δεδομένα να μεταφέρονται με τον σωστό χρονισμό από το ένα στάδιο στο άλλο και τελικά στην έξοδο.

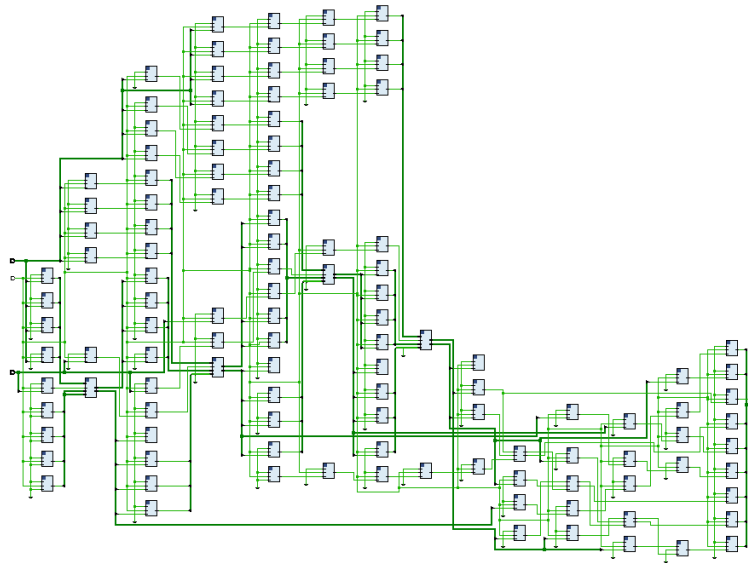
Το simulation:



Όπως διαπιστώνουμε το πρόγραμμα εκτελεί πολλαπλασιασμό με Tlatency 6 clock cycles.

Αν στο συνδυαστικό black box προσθέταμε έναν pipelined full adder 4 σταδίων, τότε το latency θα αυξανόταν σημαντικά στους $6+16=22$ κύκλοι ρολογιού.

Το **elaborated schematic**:



Τέλος αναζητούμε το **critical path**:

Name	Slack	Levels	Routes	High Fanout	From	To	Total Delay	Logic Delay	Net Delay	Requirement	Source Clock
↳ Path 1	∞	2	2	1	DFF224/q_reg/C	Cout	4.076	3.276	0.800	∞	
↳ Path 2	∞	2	2	1	dffmul22501/q_reg/C	Mul[3]	4.076	3.276	0.800	∞	
↳ Path 3	∞	2	2	1	dffmul22601/q_reg/C	Mul[4]	4.076	3.276	0.800	∞	
↳ Path 4	∞	2	2	1	dffmul1/q_reg/C	Mul[5]	4.076	3.276	0.800	∞	
↳ Path 5	∞	2	2	1	dffmua/q_reg/C	Mul[6]	4.076	3.276	0.800	∞	
↳ Path 6	∞	2	2	1	dffmulb/q_reg/C	Mul[7]	4.076	3.276	0.800	∞	
↳ Path 7	∞	2	2	1	dffmul01111/q_reg/C	Mul[0]	4.058	3.258	0.800	∞	
↳ Path 8	∞	2	2	1	dffmul0011311/q_reg/C	Mul[1]	4.058	3.258	0.800	∞	
↳ Path 9	∞	2	2	1	dffmul223301/q_reg/C	Mul[2]	4.058	3.258	0.800	∞	
↳ Path 10	∞	2	2	4	DFF_b_0/q_reg/C	DFF_factor0_0/q_reg/R	1.862	0.751	1.111	∞	

Τέλος, σημειώνουμε ότι το critical path του πολλαπλασιαστή είναι τα path 1,2,3,4,5,6 με συνολική καθυστέρηση 4.076ns.